

AFRILANG Stdlib

std/io

Bibliothèque AFRILANG ``std/io`` (niveau simple, catégorie core). Elle expose 5 fonction(s) : `readFile`, `writeFile`, `fileExi`

Import : ``import "std/io"`` puis ``use`` le module.

Utilisez ce module pour des opérations spécialisées dans son domaine. Consultez le guide PDF pour la référence complète,

Category: core | Tier: simple | Functions: 5

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/io` (niveau simple, catégorie core). Elle expose 5 fonction(s) : readFile, writeFile, fileExi

Import : `import "std/io"` puis `use` le module.

Utilisez ce module pour des opérations spécialisées dans son domaine. Consultez le guide PDF pour la référence complète,

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/io"
use io
```

Niveau : simple · Catégorie : Core & I/O

Entrées/sorties, fichiers, JSON, HTTP, collections, chemins.

3. Référence API

```
? readFile(path text) ? text
? writeFile(path text, content text) ? void
? fileExists(path text) ? bool
? readLine() ? text
? readFileAsync(path text) ? text
```

4. Exemples d'utilisation

```
import "std/io"
use io

create result = readFile(/* args */)
say result

writeFile(/* args */)

create result = fileExists(/* args */)
say result

create result = readLine(/* args */)
say result

create result = readFileAsync(/* args */)
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/io"` en tête de fichier.
? Lancez `afrilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module io

function readFile(path text) returns text
end

function writeFile(path text, content text)
end

function fileExists(path text) returns bool
end

function readLine() returns text
end
```

```
function readFileSync(path text) returns text
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/io` (tier simple, category core). Exports 5 function(s): readFile, writeFile, fileExists, readLine

Import: `import "std/io"` then `use` the module.

Use this module for specialized operations in its domain. See the PDF guide for full reference, examples and best practice

2. Installation & import

Add the module to your program:

```
import "std/io"
use io
```

Tier: simple · Category: Core & I/O
I/O, files, JSON, HTTP, collections, paths.

3. API reference

```
? readFile(path text) ? text
? writeFile(path text, content text) ? void
? fileExists(path text) ? bool
? readLine() ? text
? readFileAsync(path text) ? text
```

4. Usage examples

```
import "std/io"
use io

create result = readFile(/* args */)
say result

writeFile(/* args */)

create result = fileExists(/* args */)
say result

create result = readLine(/* args */)
say result

create result = readFileAsync(/* args */)
say result
```

5. Best practices

```
? Prefer `import "std/io"` at the top of your file.
? Run `afrilang check` before shipping.
? Combine with related stdlib modules from the same category.
? See /stdlib/ for the live source and playground demos.
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module io

function readFile(path text) returns text
end

function writeFile(path text, content text)
end

function fileExists(path text) returns bool
end

function readLine() returns text
end
```

```
function readFileSync(path text) returns text
end
end
```